

A IMPORTÂNCIA DA ATIVIDADE DE TESTE NO DESENVOLVIMENTO DE SOFTWARE

Karla Pires de Souza (FPM)

karlapsouza@hotmail.com

Angelita Moutin Segoria Gasparotto (FPM)

angelita@usp.br



A atividade de teste de software tem um objetivo de encontrar defeitos inseridos no decorrer do processo de desenvolvimento. Estima-se que o custo com correção de falhas aumenta com o avanço das fases, mesmo assim, em algumas empresas a fase de teste ainda não é executada com tempo hábil para atividade ou os testes são efetuados pelos próprios desenvolvedores dos sistemas. Este artigo tem o objetivo de apresentar conceitos de teste de software, assim como um estudo de caso realizado em um centro de desenvolvimento de software, onde se pode constatar a importância da execução dos testes por uma equipe qualificada e estruturada, tal resultado foi levantado a partir da análise de defeitos e horas trabalhados de três projetos da empresa.

Palavras-chaves: Teste de software; Qualidade; Desenvolvimento de software.

1. Introdução

A atividade de teste em várias empresas é executada pela equipe de desenvolvimento de software, que muitas vezes não possui a qualificação adequada para exercer tal tarefa. Isto ocorre, pois a atividade não é considerada tão importante no processo de construção do software. Devido à sobrecarga de trabalho e aos curtos prazos a atividade de teste é afetada e como resultado é gerado um sistema com grande quantidade de defeitos.

Estima-se que o custo com correção de falhas avança com o passar das fases de desenvolvimento de software, entretanto, mesmo com este aumento de custo, muitas empresas ainda negligenciam a fase de teste.

Este artigo tem a finalidade de listar a importância da atividade do teste no desenvolvimento de software, assim como mostrar os benefícios obtidos com a execução dos testes e apresentar alguns tipos de teste existentes. Por fim será evidenciada, através de um estudo de caso, a quantidade de defeitos evitados/corrigidos após execução dos testes.

2. Teste de software

A atividade de teste é uma das etapas do ciclo de desenvolvimento de software e tem o objetivo de relatar possíveis defeitos existentes no sistema para que estes sejam solucionados. Nesta fase verifica-se se o comportamento do sistema está de acordo com o especificado nos requisitos levantados junto ao cliente. A partir desta atividade pode-se diagnosticar o grau de qualidade do sistema. O principal objetivo da realização do teste de software é reduzir a probabilidade de ocorrência de erros quando o sistema estiver em produção (MOLINARI, 2012).

Na década de 70 a execução do teste nos softwares era feita pelos próprios desenvolvedores dos sistemas, a atividade era vista como uma tarefa secundária, sem muita importância, feita apenas se o prazo de entrega e custo do produto permitisse. Com o passar do tempo, devido à concorrência existente no mercado e ao aumento da complexidade dos sistemas, o nível de exigência por qualidade aumentou e com isso a necessidade de testes mais eficazes.

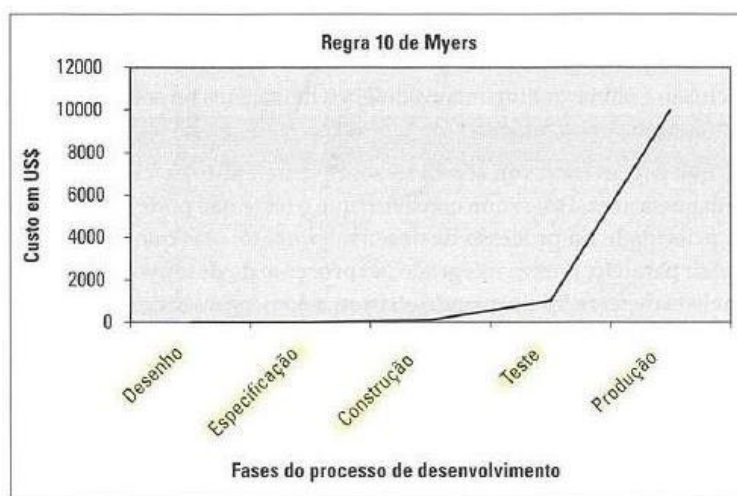
Para Pressman (2002), "teste de software é um elemento crítico da garantia de qualidade de software e representa a revisão final da especificação, projeto e geração de código". Por ser a última etapa, antes da entrega ao cliente, à fase de teste tem a responsabilidade de encontrar as falhas inseridas no decorrer do projeto.

Pressman (2002) define qualidade como a satisfação de requisitos funcionais e de desempenho explicitamente declarados, normas de desenvolvimento explicitamente documentadas e características implícitas que são esperadas em todo software desenvolvido profissionalmente. O conceito de qualidade é variável, para a equipe de desenvolvimento de software um produto possui qualidade quando se comporta conforme está descrito nos requisitos, já para o cliente um produto terá qualidade quando este atender suas necessidades. Devido ao aumento da exigência por qualidade, várias normas, metodologias ou modelos (ágil e RUP, por exemplo) e órgão reguladores como CMMI, MPS.Br e ISO foram criados.

3. Custo da não qualidade

Além de colaborar para obtenção de um produto com qualidade, a atividade de teste tem o papel fundamental na redução de custos com possíveis reparos ao sistema. Em 1979 Myers apresentou em seu livro "The Art of Software Testing" (A arte de teste de software) a Regra de 10, que constatava que quanto mais cedo descoberto e corrigido um erro, menor é o seu custo para o projeto. De acordo com (MYERS, 1979) o custo em correção cresce exponencialmente 10 vezes para cada estágio que o projeto avança, conforme apresentado no gráfico abaixo (Figura1), ou seja, defeitos encontrados nas fases iniciais do desenvolvimento do software são mais baratos de serem corrigidos do que aqueles encontrados quando o software já está em produção.

Figura 1 - Custo dos defeitos com passar das fases do projeto



Fonte: (BASTOS; RIOS; CRISTALLI; MOREIRA, 2007)

O teste de software não é uma atividade barata e fácil, normalmente o valor gasto com teste varia de 30% a 40% do valor total do projeto, dependendo das técnicas de teste que foram utilizadas e da tolerância a falhas exigidas pelo projeto (BASTOS; RIOS; CRISTALLI; MOREIRA, 2007). Como é possível verificar na imagem abaixo (Figura2) o custo com falhas é muito maior do que custo gasto com atividades de prevenção como revisões e inspeções em artefatos e código-fonte.

Figura 2 - Categoria dos custos com falhas

	Categorias dos Custos Operacionais da função Qualidade			
	Prevenção prevenir defeitos	Avaliação remover do processo os produtos defeituosos	Custos do que ocorre quando a função Qualidade falha	
			Falhas Internas ocorrem dentro da empresa	Falhas Externas ocorrem após ter sido vendido ao cliente
Custos da Qualidade (total gasto para prevenir falhas/defeitos)	5% a 15%			
Custos da Não-Qualidade (só passam a existir em consequência de falhas)		20 a 25%	65 a 70%	
	São controláveis Investimentos	Não são controláveis Perdas e Prejuízos		

Fonte: (FROTA, 1999)

Atualmente ainda existem empresas que negligenciam a atividade de teste devido ao custo e tempo exigidos pela fase. Muitas vezes os testes ainda são executados por desenvolvedores ou outro profissional que tenha conhecimento das regras de negócio do sistema. Também é comum que a fase de teste seja abreviada para que o prazo e custo do projeto sejam alcançados, consequentemente, é comum entregar ao cliente um

software com defeitos não revelados (MOLINARI, 2012). Tal atitude pode acarretar um alto custo à empresa para solucionar os defeitos encontrados após a entrega, assim como a queda de credibilidade e satisfação do cliente.

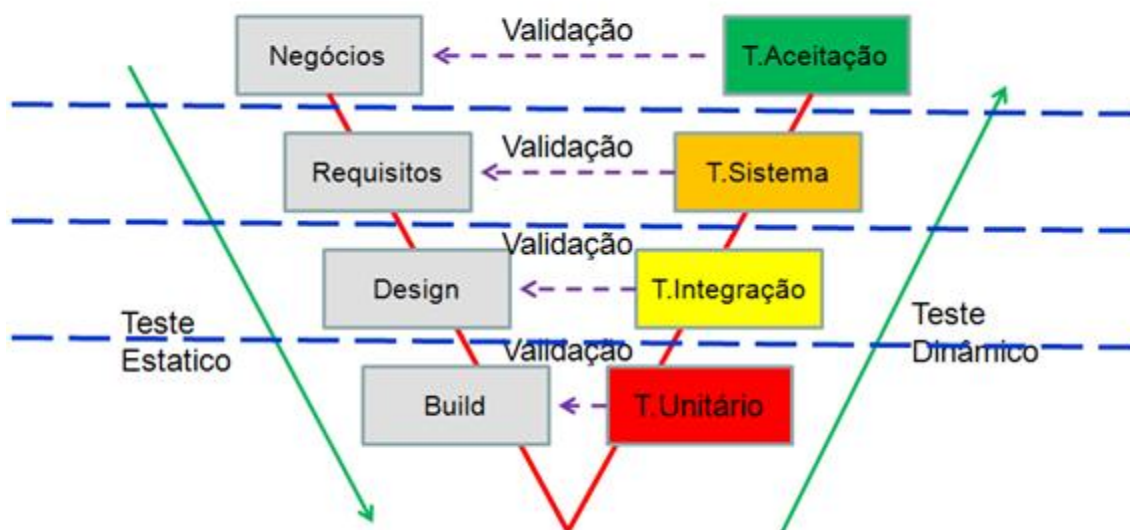
4. Verificação e validação

Para que os possíveis defeitos inseridos durante o andamento do projeto sejam encontrados o quanto antes, é importante que atividades de verificação e validação sejam executadas. O objetivo da verificação é garantir que sistema está sendo desenvolvido da maneira correta, dentro das normas e metodologia adotadas para o projeto, nesta atividade normalmente são realizadas revisões e inspeções dos artefatos e código-fonte, ou seja, uma verificação estática (SILVA, 2013).

Na validação é verificado se o sistema foi construído conforme descrito nos requisitos, esta é uma verificação dinâmica, pois é necessário navegar pelo sistema para detectar possíveis diferenças entre o sistema e os artefatos do cliente.

O modelo em V é um dos mais aceitos atualmente, ele enfatiza as atividades de verificação e validação com o objetivo de encontrar defeitos gerados durante o desenvolvimento do software e minimizar os riscos do projeto. Este modelo reforça o entendimento de que a atividade de teste não deve ser efetuada apenas no fim do projeto e sim em todo o desenvolvimento do sistema. Na Figura acima (Figura3) é possível verificar o paralelismo entre as atividades de teste (à direita) e desenvolvimento (à esquerda).

Figura 3 - Modelo em V



Fonte: (SILVA, 2013)

4.1. Níveis ou estágios de teste

Os testes podem ser divididos em quatro níveis conforme exibido na Figura 3, estes níveis definem basicamente a fase em que estes serão executados, ou seja, qual o melhor momento de executar determinado teste (BASTOS; RIOS; CRISTALLI; MOREIRA, 2007).

- a) O Teste Unitário valida a menor parte do código-fonte, neste nível de teste é verificado, por exemplo, o método de uma classe.
- b) No Teste de Integração é validado se a integração entre sistemas, componentes ou outras funcionalidades foi feita corretamente.
- c) Na fase do Teste de Sistema o software é exercitado como um todo afim de encontrar discrepâncias entre os funcionamento especificado nos requisitos e o construído.
- d) O Teste de Aceitação é realizado pelo cliente no momento da entrega do sistema, ou de parte dele, este teste é executado para validar se o sistema realiza o que solicitado.

4.2. Técnicas de teste

Atualmente existem várias formas de testar um software. Técnicas utilizadas antigamente ainda são de grande valia apesar da mudança de tecnologia e linguagem de programação, apesar destas mudanças o objetivo continua o mesmo: encontrar o maior número de defeitos antes da entrega ao cliente.

As técnicas são classificadas com base na origem das informações utilizadas para teste (código-fonte ou informações das funcionalidades do sistema). Estas técnicas abordam perspectivas diferentes do software e podem ser classificadas como: Funcional e Estrutural (NETO, 2013).

4.2.1 Técnica estrutural ou Teste de caixa branca: Nesta técnica é avaliado o comportamento interno do sistema, ou seja, o código-fonte. Esta técnica é recomendada nos níveis de Teste de Unidade e Teste de Integração, testes que normalmente ficam a cargo dos desenvolvedores do código-fonte, que possuem maior conhecimento no código-fonte. O objetivo desta é auxiliar na redução de possíveis problemas existentes nas funções ou unidades que compõem um software.

4.2.2 Técnica funcional ou Teste de caixa preta: A técnica é chamada de caixa preta, pois a estrutura interna do sistema não é considerada durante a execução destes testes. Esta técnica pode ser aplicada em todos os níveis de teste.

Dados de entrada são fornecidos, o teste é executado e o resultado obtido é comparado a um resultado esperado previamente conhecido. Haverá sucesso no teste se o resultado obtido for igual ao resultado esperado. O componente de software a ser testado pode ser um método, uma função interna, um programa, um componente, um conjunto de programas e/ou componentes ou mesmo uma funcionalidade (NETO, 2013).

Existem outras técnicas de teste como, por exemplo, o Teste de caixa cinza, que se trata de uma junção das técnicas de caixa preta e branca, que não serão apresentadas neste artigo.

4.3. Tipos de teste

Os tipos de teste que serão executados são definidos na fase de planejamento. Para realizar tal definição devem-se analisar vários fatores como: o público que utilizará o sistema, se este ficará disponível na internet, se fará integração com outros sistemas, se será acessível a deficientes visuais, se trabalhará com muitos registros cadastrados, se poderá ser acessado por vários usuários simultaneamente, entre outras informações, ou seja, devem-se verificar os

riscos do projeto, seu nível de tolerância a erros (BASTOS; RIOS; CRISTALLI; MOREIRA, 2007).

A figura abaixo apresenta os métodos, os estágios de cada método e os tipos de teste divididos por categorias:

Os tipos de teste apresentados na Figura 4, destacados em azul, são executados no nível do teste de sistema.

Figura 4 - Distribuição dos tipos de teste por categorias

Métodos	Estágios					
		Funcionalidade	Usabilidade	Confiabilidade	Desempenho	Suportabilidade
		Tipos de Teste				
Caixa Branca	Teste Unitário	Teste Funcional	Teste de Usabilidade	Teste de Estresse	Teste de Desempenho	Teste de Instalação
	Teste de Integração	Teste de Volume		Teste de Regressão	Teste de Carga	
Caixa Preta	Teste de Sistemas	Teste de Segurança		Teste de Integridade	Teste de Contenção	
	Teste de Aceitação	Teste de Acessabilidade		Teste de Estrutura		Teste de Configuração

Fonte: Adaptado de (DATASUS, 2013)

Na tabela abaixo (Quadro1) são discriminados alguns dos tipos de teste existentes e sua descrição.

Figura 5 - Tipos de teste

Tipos de teste	Descrição
Teste de Estresse	Avalia o desempenho do sistema com volume de acesso/transações acima da média esperada e em condições extremas de uso.
Teste de Performance ou Desempenho	Avalia se o sistema atende os requisitos de performance (proficiência) com volume de acesso/transações dentro do esperado. Verifica se o tempo de resposta da aplicação está conforme o esperado.
Teste de Contenção	Avalia se o sistema retorna a um status operacional após uma falha.
Teste de Segurança	Avalia o nível de segurança do sistema quanto a perfis de acesso, permissões e uso de tags html nos campos.
Teste de Regressão	Avalia se funcionalidade que estava funcionando ainda funciona após uma modificação no sistema. Todo o sistema é verificado neste teste.
Teste de Integração	Avalia se a interconexão entre aplicações ou funcionalidades funciona corretamente.
Teste de Funcionalidade	Avalia se o sistema funciona conforme descrito nos requisitos.
Teste de Usabilidade	Verifica se a navegabilidade, objetivos e padrões de tela estão conforme especificado e se atendem da melhor forma ao usuário.
Teste de Volume	Avalia o comportamento do sistema com grande quantidade de dados.
Teste de Acessibilidade	Avalia se a aplicação está navegável para um usuário com algum tipo de deficiência visual.
Teste de Estrutura	Avalia se a codificação foi feita corretamente e se todos os métodos são utilizados.
Teste de Carga	Avalia o funcionamento da aplicação com a utilização de uma grande quantidade de usuários simultâneos.
Teste de Instalação	Testar se a instalação da aplicação obteve sucesso.
Teste de Configuração	Avalia se a aplicação funciona corretamente em ambientes diferentes de hardware ou software (Ex: Navegadores diferentes).

Fonte: Adaptado de (BARBOSA, 2011)

No Quadro 2, para melhor entendimento das atividades de teste, foram listados alguns dos possíveis cenários executados em uma funcionalidade de saque em conta corrente e conta poupança. Na coluna “Exemplos de cenários de teste” são listados todos os cenários que serão verificados e então estes cenários foram divididos em Tipos de teste, ou seja, a qual tipo de teste o cenário a ser testado se enquadra.

Figura 6 - Cenários de teste para realização de saque

Figura 6 - Cenários de teste para realização de saque

Exemplos de cenários de teste	Divisão dos cenários em tipos de teste					
	Funcional	Segurança	Usabilidade	Performance		
Simular saques acima do saldo disponível.						
Simular saques com cartão vencido.	Simular saques acima do saldo disponível.	Simular saques com cartão vencido.	Avaliar se todas as telas possuem opção de ajuda: - Avaliar se mensagens são claras e objetivas. - Avaliar se padrão visual é mantido em todas as telas. - Avaliar se todas as operações possuem caminho de fuga (opção Cancelar).	Avaliar se a duração do saque dura até 30 seg. Em um universo de 5 milhões de correntistas e 100 milhões de movimentações bancárias: - Garantir que tempo de espera das operações não ultrapasse 10 seg.		
Avaliar se a duração do saque dura até 30 seg. Em universo de 5 milhões de correntistas e 100 milhões de movimentações bancárias.	Simular saque na conta poupança: - Saque acima do valor limite da conta. - Saque com valor não múltiplos das notas (EX: R\$50,50).	Avaliar se senha do cartão é solicitada antes das transações.				
Simular saque com problemas no caixa eletrônico.	- Saque com valores múltiplos das notas (Ex: R\$50).	Avaliar se senha adicional e randômica é solicitada no início da operação.				
Simular saque com impressoras dos fornecedores A, B e C.						
Avaliar se senha do cartão é solicitada antes das transações.	Carga e Concorrência	Configuração	Recuperação	Contingência		
Simular 2 saques simultâneos na mesma conta corrente.	Simular 2 saques simultâneos na mesma conta corrente (Concorrência): - Simular 10.000 saques simultâneos em contas correntes (Concorrência e Carga).	Simular saque com impressoras dos fornecedores A, B e C.	Simular saque com problemas no caixa eletrônico: - Simular saque com defeitos na impressora. - Simular saque com falhas de conexão com a central. - Simular saque com queda de energia.	- Disparar processo de instalação emergencial.		
Simular saque na conta poupança.						
Avaliar se senha adicional e randômica é solicitada no início da operação.						
Simular saques no sistema operacional Windows versões 95, 98, NT e 2000.		Simular saques no sistema operacional Windows versões 95, 98, NT e 2000.				
Avaliar se todas as telas possuem opção de ajuda.						

Fonte: Adaptado de (BARTIE, 2005)

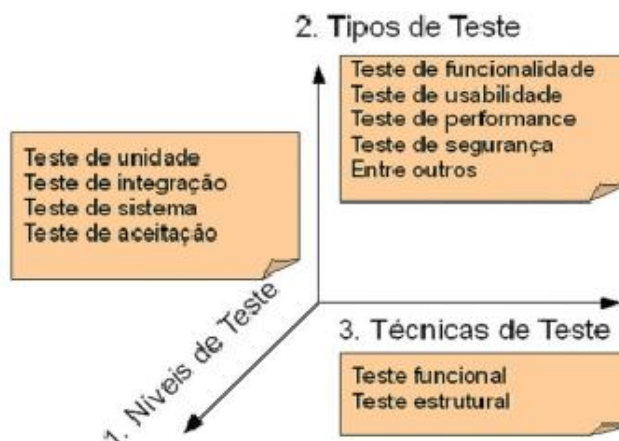
4.4. Estratégia de teste

Para elaboração de uma estratégia de teste são analisados quais os métodos, níveis e tipos de teste de serão adotados.

Na construção da estratégia de teste devem ser considerados diversos fatores como: o porte e importância do software, os requisitos, os prazos assumidos, os riscos do projeto, entre outros fatores (RIOS; MOREIRA, 2006).

A Figura 5 demonstra a relação existente entre as técnicas, níveis e tipos de teste.

Figura 7 - Estratégia de teste

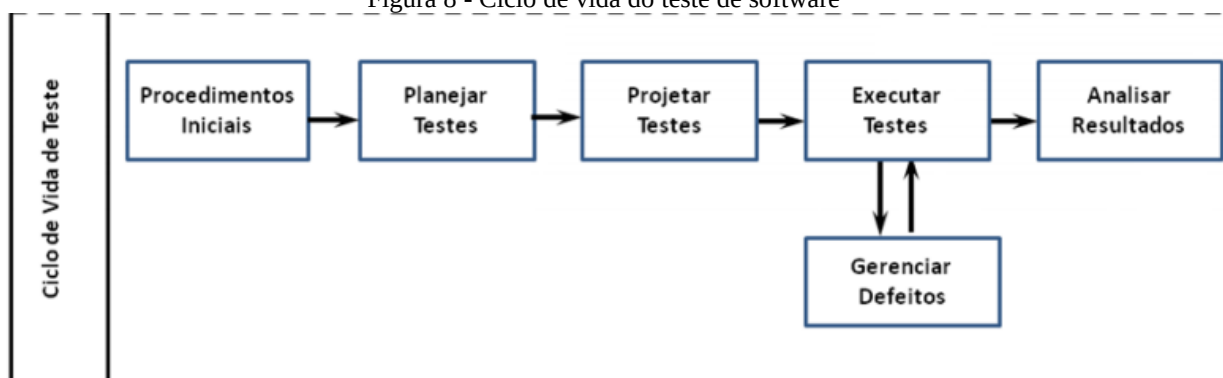


Fonte: (SILVA, 2013)

5. Ciclo de vida de teste

A Figura 6 apresenta o ciclo de vida da fase de teste. As atividades de teste serão melhor detalhadas no estudo de caso.

Figura 8 - Ciclo de vida do teste de software



Fonte: (RIOS, 2013)

Na fase de iniciação (Procedimento inicial) é feita a investigação do contexto e escopo do projeto com intuito de criar estratégias e diretrizes.

A próxima fase é a de Planejamento (Planejar Testes), neste momento são analisados os objetivos das atividades de teste. A documentação do projeto é analisada para obtenção de informações como os riscos e nível de importância do projeto para que então sejam definidos quais técnicas e tipos de testes melhor se adequam. Cada tipo de aplicação possui características específicas que devem ser consideradas no momento do planejamento e realização dos testes.

O passo seguinte é a elaboração dos casos de teste (Projetar testes) e preparação do ambiente. Após elaboração dos artefatos da fase de teste tem início a execução dos testes definidos no Plano de teste. São executados primeiro e segundo ciclo de teste (Teste e Reteste, respectivamente) e gestão e monitoramento dos defeitos registrados durante o andamento dos testes.

Por fim, ao término dos testes e correção dos defeitos é feito o levantamento das ocorrências abertas e feita a liberação do sistema ou parte dele ao cliente.

6. Estudo de caso

6.1. Informações da empresa

A empresa analisada para o estudo de caso será identificada como empresa Alfa. Esta atua no mercado de desenvolvimento de software há 23 anos. Possui filiais em vários pontos do Brasil e algumas no exterior. A Alfa tem aproximadamente 1.800 colaboradores e conquistou no decorrer destes anos as certificações ISO9001:2008, CMMI nível 3 e MPS.Br nível C.

A norma ISO-9001:2008 especifica os requisitos para um sistema de gestão da qualidade, voltado para organizações que necessitam demonstrar sua habilidade de fornecer produtos que satisfaçam os requisitos dos seus clientes, requisitos estatutários e da legislação aplicável de forma consistente e que buscam aumentar a satisfação dos seus clientes através da aplicação efetiva do sistema, incluindo processos para a sua melhoria contínua e para a garantia da conformidade com estes requisitos (CEDET, 2009).

6.2. Fase de teste

A execução dos testes tem início após construção e liberação do código-fonte, entretanto antes de exercitar o sistema é necessária a elaboração de alguns artefatos.

O primeiro artefato da fase de teste a ser elaborado é o Plano de teste, que como próprio nome sugere, é o documento que conterà todo o planejamento realizado para a execução dos testes, nele terão informações como as funcionalidades que deverão ser testadas, quais métodos e tipos de teste serão executados, as ferramentas necessárias, configuração do ambiente de teste, entre outras informações.

O caso de teste também é um dos documentos construídos antes da execução dos testes. Nele são descritos os passos a passos que devem ser executados durante a verificação do sistema. Normalmente é descrita a ação do ator e a resposta do sistema para determinado cenário. Na ilustração abaixo (Figura 7) é possível verificar com mais detalhes como são elaborados os casos de teste. Na primeira coluna há uma breve descrição do cenário que será verificado, na segunda coluna são descritos os passos que o ator (usuário do sistema) deve executar e na terceira coluna é informado o resultado esperado do sistema para a ação executada. A quarta coluna indicará a situação do cenário, se o sistema se comportou conforme o indicado nos requisitos.

Figura 9 - Exemplo de caso de teste

Caso de Teste	Especificação	Ação do Ator	Resultados Esperados	Resultado Obtido TESTE	Resultado Obtido RETESTE	ID - Defeito
Cenário 4.1: Visualizar dados do usuário.						
Pré-condição:						
1 - O Sistema de Agendamento deve estar operacional.						
2 - O Usuário do Sistema de Agendamento deve estar autenticado no Sistema, e possuir perfil de Administrador, Central de Atendimento ou Posto de Atendimento.						
3 - O Usuário do Sistema de Agendamento já deve ter realizado uma consulta que tenha apresentado no mínimo um resultado.						
CT004.01	Validar início da funcionalidade de visualizar dados do usuário	Após arealização de uma consulta e apresentação dos resultados o Usuário do Sistema de Agendamento seleciona a opção "Visualizar" de um dos registros apresentados.	O Sistema identifica os dados referentes ao Usuário do Sistema de Agendamento desejado e exibe tela para visualização dos dados do cliente de acordo com o perfil do usuário.	NE	NE	
CT004.02	Validar campos de visualização do registro	O Usuário do Sistema de Agendamento verifica os campos de visualização do registro	O teste se desvia para a aba de teste: "Tela2"	Aprovado	Aprovado	
CT004.03	Validar tela de visualização dos dados do cliente com perfil do tipo: "Pessoa Física" .	O Usuário do Sistema de Agendamento verifica tela de visualização dos dados para usuário com perfil pessoa física	O Sistema exibe a tela para visualização dos dados do cliente de acordo com: "Tela Cliente Pessoa Física"	Reprovado	Reprovado	

Fonte: Elaborado pelo autor

Após a elaboração dos artefatos e liberação do caso de uso para teste inicia-se a primeira bateria de testes, neste momento são exercitados os tipos de teste definidos na fase de planejamento. A equipe de teste percorre os cenários descritos nos casos de teste e caso encontre algum defeito o registra em uma ferramenta de gestão de ocorrência.

Os defeitos são abertos para o desenvolvedor do caso de uso. Este verifica se o defeito realmente ocorre e em caso positivo realiza a correção do código-fonte. Após correção o desenvolvedor libera a modificação para uma nova validação.

Neste momento o testador que cadastrou a ocorrência verifica se o defeito foi eliminado e caso esteja correto a ocorrência é fechada.

Após execução da primeira bateria de testes é iniciado o reteste, onde ocorre uma nova validação da funcionalidade ou caso de uso por outro testador. A execução de duas baterias de teste por recursos diferente é importante, pois permite que o sistema seja analisado por pessoas com experiências profissionais, conhecimentos e características diferentes. Na etapa de reteste os defeitos encontrados são registrados e corrigidos assim como ocorrido na primeira etapa do teste.

Após execução do Teste e Reteste é iniciado o Teste Regressivo. Nesta etapa o intuito é encontrar possíveis defeitos que foram gerados devido às correções. Neste momento a quantidade de defeitos encontrada normalmente é baixa e o sistema é totalmente percorrido, ou seja, caso o sistema tenha casos de uso que já foram entregues, no Teste Regressivo serão verificados os casos de uso que serão e os que já foram entregues.

Com o término dos testes, nos projetos tradicionais, ou seja, os que não usam a metodologia ágil, é elaborado o Sumário de Avaliação de Teste. Neste artefato é listada a quantidade de defeitos por caso de uso, a quantidade de cenários por caso de teste, os defeitos abertos, os fechados, os defeitos satisfatórios (que realmente ocorriam) e os insatisfatórios (abertos indevidamente, por algum engano do testador) e os responsáveis pela execução dos testes.

Ao finalizar os três ciclos de teste e correção de todos os defeitos encontrados o sistema é disponibilizado ao cliente, para o teste de aceitação.

Para evidenciar os impactos gerados no sistema após a fase teste foram analisados 3 projetos, conforme demonstrado na tabela abaixo.

Tabela 1. Horas de teste e defeitos por projeto

Projetos	Entregas	Total horas projeto	Horas de teste	Total de horas teste	Quantidade de dias de teste*	Total de dias de teste*	Por entregas (%)	Total (%)
Projeto1	1	754,25	133,5	332,25	16,69	41,53	17,70%	44,05%
	2		198,75		24,84		26,35%	
Projeto2	1	18557,47	207,77	1017,3	25,97	127,16	1,12%	5,48%
	2		175		21,88		0,94%	
	3		392,53		49,07		2,12%	
	4		242		30,25		1,30%	
Projeto3	1	3100	115	597,5	14,38	74,69	3,71%	19,27%
	2		166,5		20,81		5,37%	
	3		151,5		18,94		4,89%	
	4		164,5		20,56		5,31%	

*Quantidade de dias calculada com 8 horas diárias de trabalho de um profissional

Fonte: Elaborado pelo autor

Tabela 2. Quantidade de defeitos por projeto.

Projetos	Quantidade de defeitos internos	Quantidade de defeitos internos indevidos	Quantidade de defeitos externos	Quantidade de defeitos externos indevidos	Quantidade de defeitos internos	Quantidade de defeitos internos indevidos
Projeto1	85	0	0	0	85	0
Projeto2	1060	15	143	10	1060	15
Projeto3	273	5	17	6	273	5

Fonte: Elaborado pelo autor

Foram coletadas informações como: Quantidade total de horas gastas no projeto, Quantidade de horas gastas com teste e Quantidade de defeitos de cada projeto.

Com base nos dados coletados e apresentados na Tabela1 é possível verificar que os Projetos 2 e 3, que são projetos que seguem a processo tradicional, qual é baseado na metodologia

RUP, o tempo gasto com teste são inferiores a 20% do projeto. Para o Projeto 1 foi adotado a metodologia ágil, deste modo, devido a menor necessidade de tempo gasto com elaboração de artefatos, torna-se possível destinar mais horas a atividade de teste.

Na Tabela 2 pode-se perceber que sem a execução dos testes o sistema seria entregue com grande quantidade de defeitos nos três projetos analisados. O projeto 1, onde foi possível destinar quantidade maior de tempo para as atividades de teste a quantidade de defeitos externos, ou seja, abertos pelo cliente, foi zero.

7. Considerações Finais

O teste de software é uma das atividades mais custosas do processo de desenvolvimento de software, pois pode envolver uma quantidade significativa dos recursos de um projeto. O rigor e o custo associado a esta atividade dependem principalmente da criticidade da aplicação a ser desenvolvida. Diferentes categorias de aplicações requerem uma preocupação diferenciada com as atividades de teste.

Realizar testes não consiste simplesmente na geração e execução de casos de teste, mas envolvem também questões de planejamento, gerenciamento e análise de resultados.

Neste artigo foram apresentados alguns conceitos sobre teste de software e o objetivo principal foi apresentar a importância da fase de teste para a entrega de um produto com maior qualidade. Entretanto, para a entrega ocorra conforme o esperado é necessário planejamento, o envolvimento de profissionais capacitados e tempo hábil para execução dos testes. Também é importante destacar que a execução dos testes deve ocorrer em todo o processo de desenvolvimento do sistema.

REFERÊNCIAS

BARBOSA, Filipe B.; TORRES, Isabelle V. **O Teste de Software no Mercado de Trabalho**, 2011. Disponível em: <<http://revista.faculdadeprojecao.edu.br/revista/index.php/projecao2/article/viewFile/82/70>>. Acessado em: 09 mar. 2013.

BARTIE, Alexandre. **Categorias de Teste de Software**, 2005. Disponível em: <<http://imasters.com.br/artigo/3648/software/categorias-de-testes-de-software/>>. Acessado em: 09 mar. 2013.

BASTOS Aderson; RIOS Emerson; CRISTALLI Ricardo; MOREIRA Trayahú. **Base de conhecimento em teste de software**. 2.ed. São Paulo: Martins, 2007, 263 p.

CEDET – Centro de Desenvolvimento Profissional e Tecnológico, 2009. Disponível em:
<<http://www.cedet.com.br/index.php?/O-que-e/Gestao-da-Qualidade/iso-90002008.html>>. Acessado em: 23 fev. 2013.

DATASUS, 2013. Disponível em: <<http://pts.datasus.gov.br/PTS/default.php?area=0407>>. Acessado em: 09 mar. 2013.

FROTA, Álvaro. **O Barato Sai Caro!** Como reduzir custos através da qualidade. São Paulo: Qualitymark, 1999, 166 p.

MOLINARI Leonardo. **Teste de Software** – Produzindo sistema melhores e mais confiáveis. 4 ed. São Paulo: Editora Érica Ltda, 2012, 228 p.

MYERS, Glenford J. **The art of software testing**. New York: John Wiley & Sons, 1979, 177p.

NETO, Arilo C. D, 2013. **Revista Engenharia de Software**. Disponível em:
<<http://www.devmedia.com.br/artigo-engenharia-de-software-introducao-a-teste-de-software/8035>>. Acessado em: 04 mar. 2013.

PRESSMAN, R. S. **Engenharia de Software**. 5. ed. Rio de Janeiro: McGraw-Hill, 2002, 843 p.

RIOS, Emerson, 2013. **Fundamentos de teste de software**. Disponível em:
<<http://www.emersonrios.eti.br/Artigos/Fund%201.pdf>>. Acessado em: 09 mar. 2013.

RIOS, Emerson; MOREIRA, Trayahú. **Teste de Software**. 2.ed. São Paulo: Alta Book, 2006, 232 p.

SILVA, Fernando R, 2013. **Revista Engenharia de Software**. Disponível em:
<<http://www.devmedia.com.br/testes-de-software-niveis-de-testes/22282>>. Acessado em: 04 mar. 2013.